

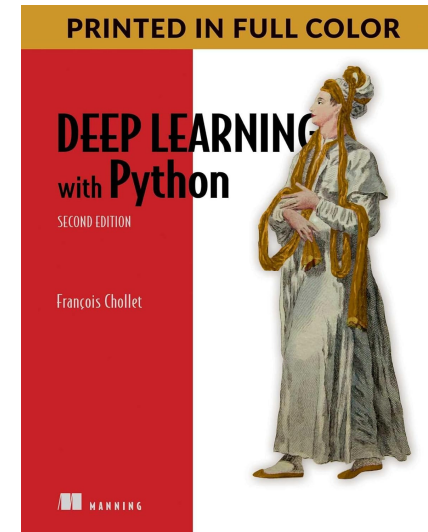
CS5900

Application of Deep Learning DL for Text (Especially **Transformer**)

Instructor: Dr. Zesheng Chen

PURDUE
UNIVERSITY
FORT WAYNE

Reference



2

PURDUE
UNIVERSITY
FORT WAYNE

Outline

- Natural Language Processing: The Bird's Eye View
- Preparing Text Data
- Two Approaches for Representing Groups of Words: Sets and Sequences
- The Transformer Architecture
- Beyond Text Classification: Sequence-to-sequence

3

PURDUE
UNIVERSITY
FORT WAYNE

Natural Language Characteristics

- Natural language is **messy**!
 - ☐ Ambiguous
 - ☐ Chaotic
 - ☐ Sprawling
 - ☐ Constantly in flux

4



Approaches to Natural Language Processing (NLP)

- Rule based
 - Such as, “rule set of English”
 - Applied linguistics
 - It is **not** easily pliable to formalization
- Machine-Learning based
 - Decision tree
 - Logistic regression
 - RNN (LSTM)
 - Transformer

5



Goal of Modern NLP

- Is the goal of modern NLP to *understand* language?
 - **No!**
- The goal is to use machine learning and large datasets to give computers to ingest a piece of language as input and return something useful, such as sentiment analysis.
 - NLP is **pattern recognition** applied to words, sentences, and paragraphs.

6



NLP Applications

- Text Classification
 - E.g., “What’s the topic of this text?”
- Content filtering
 - E.g., “Does this text contain abuse?”
- Sentiment analysis
 - E.g., “Does this text sound positive or negative?”
- Language modeling
 - E.g., “What should be the next word in this incomplete sentence?”
- Translation
 - E.g., “How would you say this in German?”
- Summarization
 - E.g., “How would you summarize this article in one paragraph?”
- Etc.

7



Outline

- Natural Language Processing: The Bird’s Eye View
- **Preparing Text Data**
- Two Approaches for Representing Groups of Words: Sets and Sequences
- The Transformer Architecture
- Beyond Text Classification: Sequence-to-sequence

8

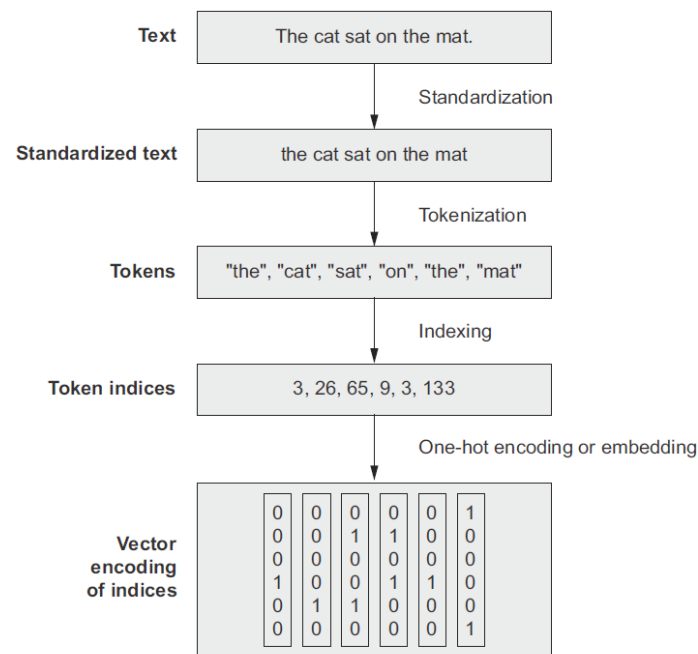


Preparing Text Data

■ **Vectorizing** text is the process of transforming **text** into **numeric** tensors.

- DL models, being differentiable functions, can **only** process numeric tensors.
- They **cannot** take raw text as input.

9



10



Three Main Steps for Text Vectorization

■ Text Standardization

- **Standardize** the text to make it easier to process, such as by converting it to lowercase or removing punctuation.

■ Text Splitting (Tokenization)

- **Split** the text into units (called tokens), such as characters, words, or groups of words.

■ Vocabulary Indexing

- Convert each such token into a numerical vector. This will usually involve first **indexing** all tokens present in the data.

11



Text Standardization

■ An example:

- "sunset came. i was staring at the Mexico sky. Isn't nature splendid??"
- "Sunset came; I stared at the México sky. Isn't nature splendid?"

■ Text standardization is a basic form of **feature engineering**.

- Aim to **erase encoding differences** that you don't want your model to have to deal with.

12

Three Standardization Schemes

- One of the **simplest** and **most** widespread standardization schemes is “**convert to lowercase and remove punctuation characters.**”
 - “sunset came i was staring at the mexico sky isnt nature splendid”
 - “sunset came i stared at the méxico sky isnt nature splendid”
- Another common transformation is to **convert special characters to a standard form.**
 - Such as replacing “é” with “e,” “æ” with “ae,” and so on. Our token “méxico” would then become “mexico”.

13

Three Standardization Schemes

■ Stemming

- A much **more advanced** standardization pattern
- **More rarely used** in a machine learning context
- Convert **variations** of a term (such as different conjugated forms of a verb) into a single shared representation
- For example, turn “caught” and “been catching” into “[catch]” or “cats” into “[cat]”
 - “sunset came i [stare] at the mexico sky isnt nature splendid”

14

Possible Issues for Text Standardization

- Standardization may also **erase** some amount of **information**, so always keep the **context** in mind.
- For instance, if you’re writing a model that extracts questions from **interview articles**, it should definitely treat “**?**” as a separate token instead of dropping it, because it’s a **useful** signal for this specific task.

15

Text Splitting (Tokenization)

- Break the text up into **units** to be vectorized.
- **Three** different ways
 - **Word-level** tokenization
 - Tokens are **space-separated** (or **punctuation-separated**) substrings.
 - A variant of this is to further split words into **subwords** when applicable—for instance, treating “staring” as “star+ing” or “called” as “call+ed.”
 - **N-gram** tokenization
 - Tokens are **groups of N consecutive words**.
 - For instance, “the cat” or “he was” would be 2-gram tokens (also called bigrams).

16

Text Splitting (Tokenization)

- **Character-level** tokenization
 - Each **character** is its own token.
 - In practice, this scheme is **rarely** used.
 - Only really see it in **specialized contexts**, like text generation or speech recognition.

17

Two Kinds of Text-Processing Models

- **Sequence** models
 - Care about word order
 - Use **word-level** tokenization
- **Bag-of-words** models
 - Treat input words as a set
 - Discard words' original order
 - Use **N-gram** tokenization

18

Vocabulary Indexing

- What is **vocabulary**?
 - All **terms** found in the training data.
- What is **vocabulary indexing**?
 - Assign a unique **integer** to each entry in the vocabulary.
- How to **implement** vocabulary indexing?

```

vocabulary = {}
for text in dataset:
    text = standardize(text)
    tokens = tokenize(text)
    for token in tokens:
        if token not in vocabulary:
            vocabulary[token] = len(vocabulary)

```

19

Details of Vocabulary Indexing

- It is common to restrict the vocabulary to only the top 20,000 or 30,000 **most common** words found in the training data. Why?
 - Avoid the issue of **overfitting**.
- When look up a new token in our vocabulary index, it may **not** necessarily exist. How to solve it?
 - Use **OOV** ("out of vocabulary") index.
 - It is usually index 1, which responds to "[UNK]" (i.e., "**OOV token**").
- What is index 0?
 - It is reserved for **mask token**., which tells us "ignore me, I'm not a word."
 - It is particularly used to **pad** sequence data.

20



Code Implementation

- Pure Python Code
- Using the *TextVectorization* Layer

21



Outline

- Natural Language Processing: The Bird's Eye View
- Preparing Text Data
- Two Approaches for Representing Groups of Words: Sets and Sequences
- The Transformer Architecture
- Beyond Text Classification: Sequence-to-sequence

22



Word Order

- Words in a sentence do **NOT** have a natural, canonical **order**. Why?
 - **Different** languages order **similar** words in very **different** ways.
 - For instance, the **sentence structure** of English is quite different from that of Japanese.
 - Within a given language, typically say the **same** thing in **different** ways by **reshuffling** the words a bit.
 - If fully **randomize** the words in a **short** sentence, you can still largely figure out what it was saying
 - Though in many cases significant **ambiguity** arises

23



NLP Architectures

- How to represent word order is the **pivotal** question for NLP architectures.
- **Two** popular models:
 1. Bag-of-words models
 - **Discard** word order
 - Treat text as an **unordered set** of words

24

NLP Architectures

2. Sequence models

■ RNN

- Process words **strictly** in the order in which they appear

■ Transformer

- **Hybrid** approach
- Technically order-agnostic
- Injects **word-position** information into the representations it processes
- Still being **order-aware**

25

Example

■ IMDB movie reviews data

- 25,000 text files for training dataset (50% positive)
- 25,000 text files for test dataset

■ Sentiment analysis problem

- Baseline?

26

Models

■ Bag-of-words model

- Single words (unigrams) with binary encoding
- Bigrams with binary encoding
- Bigrams with TF-IDF encoding

■ Sequence model

- One-hot encoding
- Word embeddings

Code Example

27

TF-IDF Encoding

- Count how many **times** each word or N-gram occurs.
- The more a given term appears in a document, the more **important** that term is for understanding what the document is about.
 - Term frequency and document frequency

```
def tfidf(term, document, dataset):
    term_freq = document.count(term)
    doc_freq = math.log(sum(doc.count(term) for doc in dataset) + 1)
    return term_freq / doc_freq
```

28

Sequence Model

- Instead of manually crafting order-based features, expose the model to **raw** word sequences and let it figure out such features **on its own**.
- Examples of sequence models
 - 1D convnet
 - RNN
 - Transformer
- First attempt: **one-hot encoding**

29

Word Embeddings

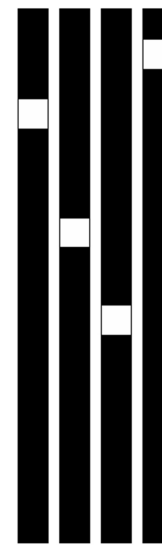
- **Assumption** used in one-hot encoding
 - The different tokens are all **independent** from each other.
 - One-hot vectors are all **orthogonal** to one another.
 - In the case of words, that assumption is clearly **wrong**.
- Words form a **structured** space: they share **information** with each other.
 - For example, words “movie” and “film” are **interchangeable** in most sentences.

30

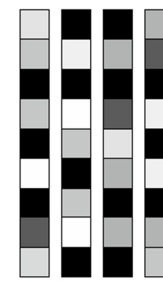
Word Embeddings

- Intuition: the **geometric** relationship between two word vectors should reflect the **semantic** relationship between these words.
- **Word embeddings** are vector **representations** of words that map human language into a **structured** geometric space.
- One-hot encoding vs. word embeddings
 - Binary vs. floating-point
 - Sparse vs. dense
 - High-dimension vs. low-dimension

31



One-hot word vectors:
- Sparse
- High-dimensional
- Hardcoded

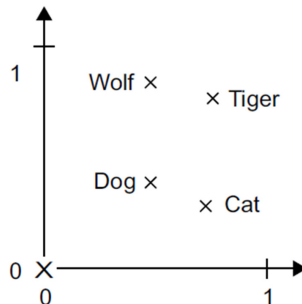


Word embeddings:
- Dense
- Lower-dimensional
- Learned from data

32

Word Embeddings

- Word embedding **structure** is learned from data.
- **Similar** words get embedded in **close** locations.
- Specific **directions** in the embedding space are meaningful.



33

Two Ways to Obtain Word Embeddings

1. **Learn** word embeddings **jointly** with the main task.
 - Start with **random** word vectors and then learn word vectors in the **same** way as learn the weights of a neural network.
2. **Pretrained** word embeddings.
 - Load into your model word embeddings that were **precomputed** using a **different** machine learning task.

34

Learning Word Embedding with Embedding Layer

- What makes a **good** word-embedding space depends heavily on your **task**.
 - English-language movie-review sentiment-analysis
 - English-language legal-document classification
- It is **reasonable** to learn a **new** embedding space with every **new** task.

```
embedding_layer = layers.Embedding(input_dim=max_tokens, output_dim=256)
```

The Embedding layer takes at least two arguments: the number of possible tokens and the dimensionality of the embeddings (here, 256).

Word index → Embedding layer → Corresponding word vector

35

Using Pretrained Word Embeddings

- Transfer learning!
- **Reuse** features learned on a different task.
- Two **popular** word embeddings
 - Word2Vec (<https://code.google.com/archive/p/word2vec/>)
 - Global Vectors for Word Representation (GloVe, <https://nlp.stanford.edu/projects/glove>)
- Apply **GloVe** in the example.

36

Outline

- Natural Language Processing: The Bird's Eye View
- Preparing Text Data
- Two Approaches for Representing Groups of Words: Sets and Sequences
- **The Transformer Architecture**
- Beyond Text Classification: Sequence-to-sequence

37

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

- <https://arxiv.org/abs/1706.03762>

38

Transformer Architecture

- Neural **attention**
 - A **simple** mechanism
 - Used to build powerful **sequence** models
 - Does **not** feature any recurrent layers or convolution layers
 - One of **most** influential ideas in deep learning
 - **Beyond** NLP
 - For example, **Vision Transformer (ViT)**
 - https://github.com/google-research/vision_transformer

39

Self-attention

- A **simple** yet **powerful** idea: **not** all input information seen by a model is **equally** important to the task at hand.
 - Pay **more** attention to some features
 - Pay **less** attention to other features
- Does that sound familiar? Two **similar** concepts:
 - **Max pooling** in convnets (“all or nothing”)
 - **Keep** the most important feature and **discard** the rest
 - **TF-IDF normalization** assigns importance scores to tokens (continuous form of attention)

40

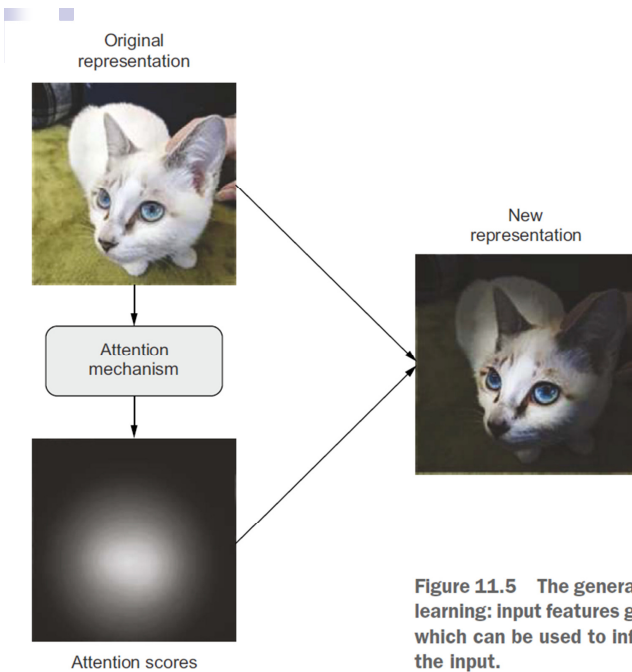
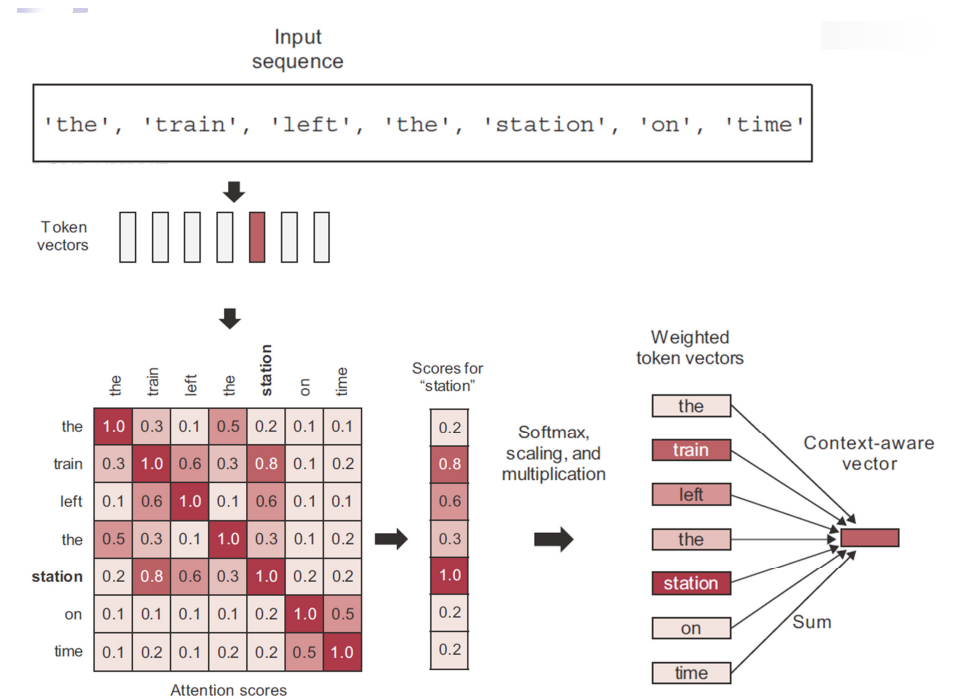


Figure 11.5 The general concept of “attention” in deep learning: input features get assigned “attention scores,” which can be used to inform the next representation of the input.



PURDUE UNIVERSITY FORT WAYNE NumPy-like Pseudocode

```
def self_attention(input_sequence):
    output = np.zeros(shape=input_sequence.shape)
    for i, pivot_vector in enumerate(input_sequence):
        for j, vector in enumerate(input_sequence):
            scores[j] = np.dot(pivot_vector, vector.T)
            scores /= np.sqrt(input_sequence.shape[1])
            scores = softmax(scores)
            new_pivot_representation = np.zeros(shape=pivot_vector.shape)
            for j, vector in enumerate(input_sequence):
                new_pivot_representation += vector * scores[j]
            output[i] = new_pivot_representation
    return output
```

Iterate over each token in the input sequence.

Compute the dot product (attention score) between the token and every other token.

Scale by a normalization factor, and apply a softmax.

Take the sum of all tokens weighted by the attention scores.

That sum is our output.

softmax function:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

PURDUE UNIVERSITY FORT WAYNE Keras MultiHeadAttention Layer

```
num_heads = 4
embed_dim = 256
mha_layer = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
outputs = mha_layer(inputs, inputs, inputs)
```

■ Questions:

- Why passing the inputs to the layer **three** times? That seems **redundant**.
- What are these “**multiple heads**” referring to?



Generalized Self-Attention: The Query-Key-Value Model

- A Transformer is a **sequence-to-sequence** model

- Convert one sequence to another
- E.g., machine translation

- **Self-attention** mechanism performs:

`outputs = sum(inputs * pairwise_scores(inputs, inputs))`

↑
C

↑
A

↑
B

- This means “for each token in inputs (A), compute how much the token is **related** to every token in inputs (B), and use these scores to **weight** a sum of tokens from inputs (C).”

45



Generalized Self-Attention: The Query-Key-Value Model

- Generalize the model to “for each element in the **query**, compute how much the element is related to every **key**, and use these scores to weight a sum of **values**”:

`outputs = sum(values * pairwise_scores(query, keys))`

- Terminology comes from **search engines** and **recommender systems**.

46

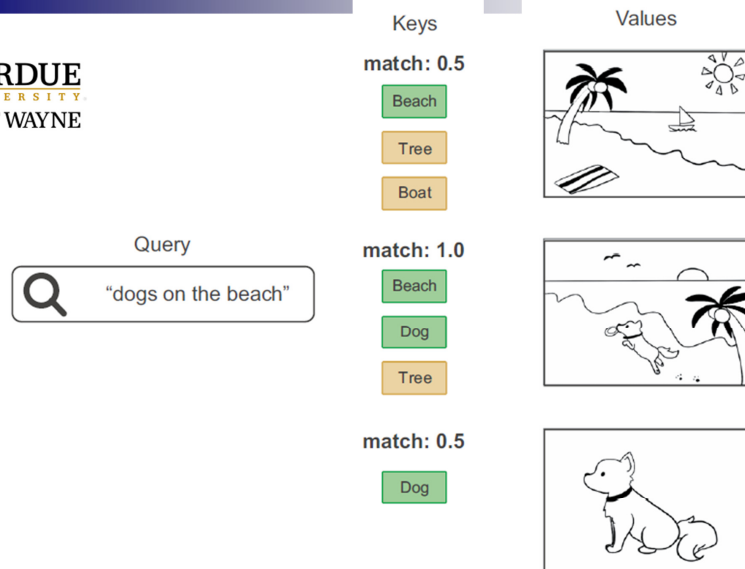


Figure 11.7 Retrieving images from a database: the “query” is compared to a set of “keys,” and the match scores are used to rank “values” (images).

47

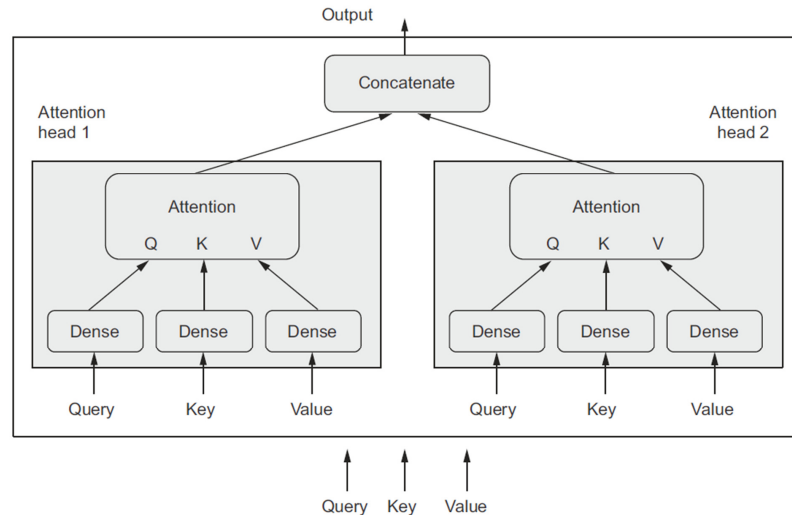


Transformer-Style Attention

- **Query**: reference sequence describes something you’re looking for.
- **Value**: a body of knowledge that you’re trying to extract information from.
- **Key**: each value is assigned a key that describes the value in a format that can be readily compared to a query.
- Machine translation example
 - Query: target sequence
 - Key and value: source sequence

48

Multi-head Attention



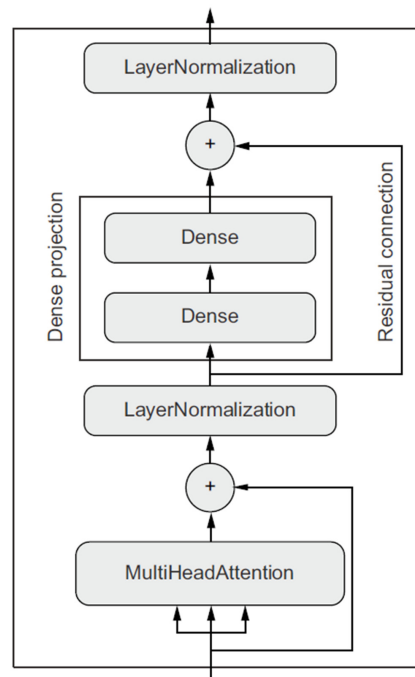
49

Multi-head Attention

- The output space of the self-attention layer gets **factored** into a set of **independent** subspaces, learned **separately**.
 - Each subspace is called a “**head**”
 - The initial query, key, and value are sent through three **independent** sets of **dense** projections
 - Results in three **separate** vectors
 - Three outputs are **concatenated** back to a single output sequence
- Similar in principle to **depthwise separable convolutions**.

50

Transformer Encoder



51

LayerNormalization vs. BatchNormalization

- BatchNormalization **doesn't** work well for sequence data.
- LayerNormalization layer normalizes each sequence **independently** from other sequences in the batch.

```
def layer_normalization(batch_of_sequences):
    mean = np.mean(batch_of_sequences, keepdims=True, axis=-1)
    variance = np.var(batch_of_sequences, keepdims=True, axis=-1)
    return (batch_of_sequences - mean) / variance
```

Input shape: (batch_size, sequence_length, embedding_dim)

To compute mean and variance, we only pool data over the last axis (axis -1).

```
def batch_normalization(batch_of_images):
    mean = np.mean(batch_of_images, keepdims=True, axis=(0, 1, 2))
    variance = np.var(batch_of_images, keepdims=True, axis=(0, 1, 2))
    return (batch_of_images - mean) / variance
```

Input shape: (batch_size, height, width, channels)

Pool data over the batch axis (axis 0), which creates interactions between samples in a batch.



APPLY TRANSFORMER ENCODER FOR TEXT CLASSIFICATION

53



Comparing NLP Models

	Word order awareness	Context awareness (cross-words interactions)
Bag-of-unigrams	No	No
Bag-of-bigrams	Very limited	No
RNN	Yes	No
Self-attention	No	Yes
Transformer	Yes	Yes

54



Using Positional Encoding to Re-inject Order Information

- Positional encoding
 - Give the model access to **word-order** information
 - Add the word's **position** in the sentence to each word embedding
- Word embedding two components
 - **Word vector**: represents the word **independently** of any specific context
 - **Position vector**: represents the **position** of the word in the current sentence

55



How to Add Position To Embedding Vector?

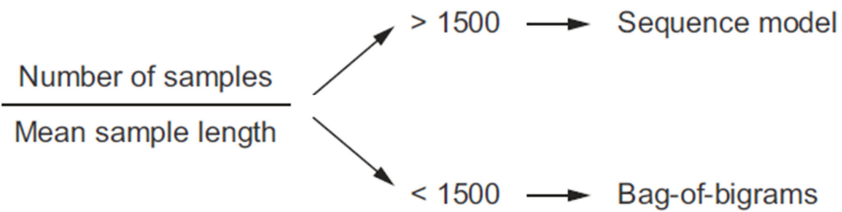
- Concatenate the word's position as a new axis to its embedding vector
 - Not working!
 - Neural networks do not like large input values
- Apply cosine function to map positions to values in the range $[-1, 1]$
 - Proposed in original "Attention is all you need" paper
- Positional embedding
 - The same way to learn to embed word indices

56



Sequence Models Vs. Bag-of-Words Models

- A simple heuristic for selecting a text-classification model:



57



Outline

- Natural Language Processing: The Bird's Eye View
- Preparing Text Data
- Two Approaches for Representing Groups of Words: Sets and Sequences
- The Transformer Architecture
- **Beyond Text Classification: Sequence-to-sequence**

58



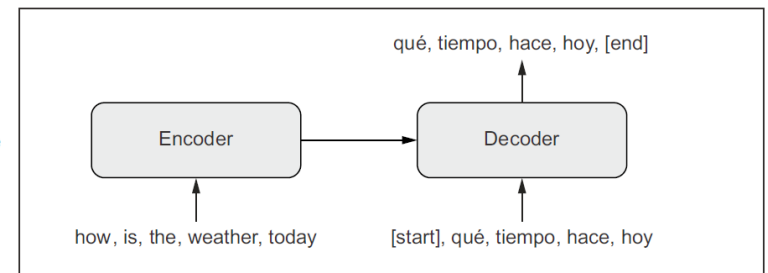
Sequence-to-sequence Model

- Takes a **sequence** as input (often a sentence or paragraph) and translates it into a **different sequence**
- **Applications** of NLP
 - Machine translation
 - Text summarization
 - Question answering
 - Chatbots
 - Text generation
 -

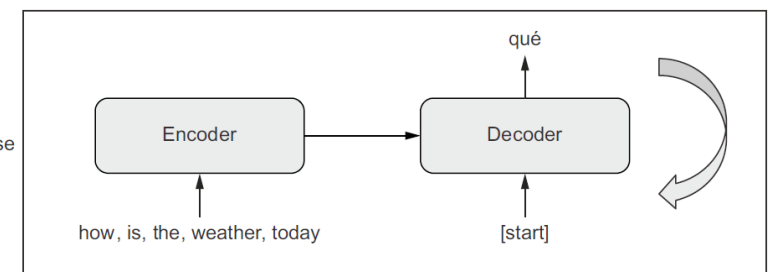
59



Training phase



Inference phase





A Machine Translation Example

■ English-to-Spanish translation

■ Steps:

- Download the dataset
- Prepare the datasets for the translation task
- Build the model
 - RNNs
 - Transformer

61



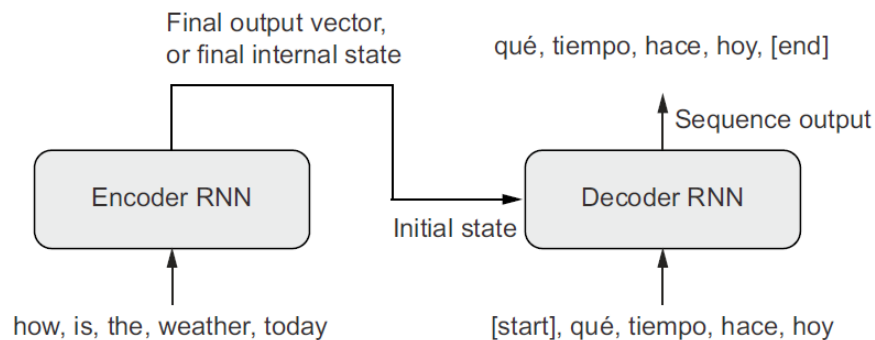
Sequence-to-sequence Learning with RNNs

- Google Translate circa 2017 was powered by a stack of seven large LSTM layers.
- The **simplest, naive** way to use RNNs to turn a sequence into another sequence is to keep the output of the RNN at each time step.
 - **Not** work
 - The target sequence must always be the **same** length as the source sequence
 - Due to the step-by-step nature of RNNs, the model will **only** be looking at tokens 0...N in the source sequence in order to predict token N in the target sequence.

62



Sequence-to-sequence Learning with RNNs



63



Fundamental Limitations of RNN Approach

- RNNs have **trouble** dealing with very **long** sequences, since they tend to progressively forget about the past.
- RNN-based models **can't** hold onto long-term context, which can be essential for translating **long** documents.

64



Sequence-to-sequence Learning with Transformer

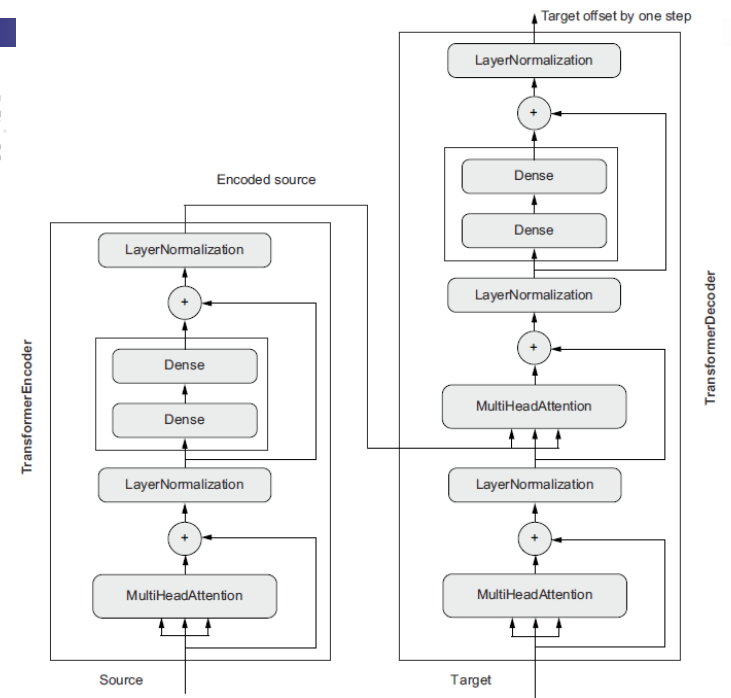
■ Transformer **encoder**

- Uses **self-attention** to produce context-aware representations of each token in an input sequence.
- Keeps the encoded representation in a sequence of **context-aware embedding** vectors format.

■ Transformer **decoder**

- Reads tokens $0 \dots N$ in the target sequence and tries to predict token $N+1$.
- The **target** sequence serves as an attention “**query**” whereas the **source** sequence plays the roles of both **keys** and **values**.

65



66



Summary

■ Two kinds of NLP models

- **Bag-of-words** models and **sequence** models

■ Word embeddings

- Vector spaces where **semantic** relationships between words are modeled as **distance** relationships between vectors that represent those words

■ Sequence-to-sequence model

- Made of an **encoder** and a **decoder**

■ Transformer architecture

- Yields **excellent** results on sequence-to-sequence tasks

67

